

HTTP e Arquitetura da Web

Programação Web I

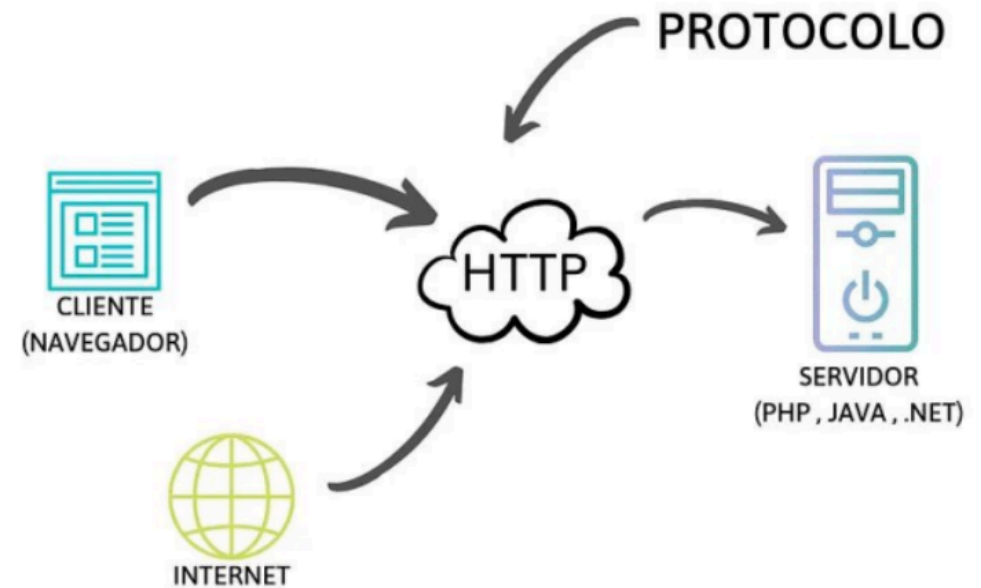
Objetivos da Aula

- Compreender o funcionamento da Web
- Entender o protocolo HTTP
- Analisar o modelo cliente-servidor
- Identificar estrutura de mensagens HTTP
- Diferenciar tipos de conexões HTTP

Contextualização

- A Web é baseada em **comunicação entre sistemas**
- Navegador (cliente)  Servidor Web
- Essa comunicação ocorre via **HTTP**

 Base para desenvolvimento **backend**
e **APIs REST**



O que é HTTP?

- **HyperText Transfer Protocol**
- Protocolo da **camada de aplicação**
- Modelo **cliente-servidor**
- Define:
 - Formato das mensagens
 - Regras de comunicação

Arquitetura da Web

Modelo Cliente-Servidor

- **Cliente** → faz requisições
- **Servidor** → responde com recursos

Exemplo:

- Navegador requisita uma página HTML
- Servidor retorna `HTML + CSS + JS`

Lado Cliente

 Ponto de partida da interação do usuário com a Web.

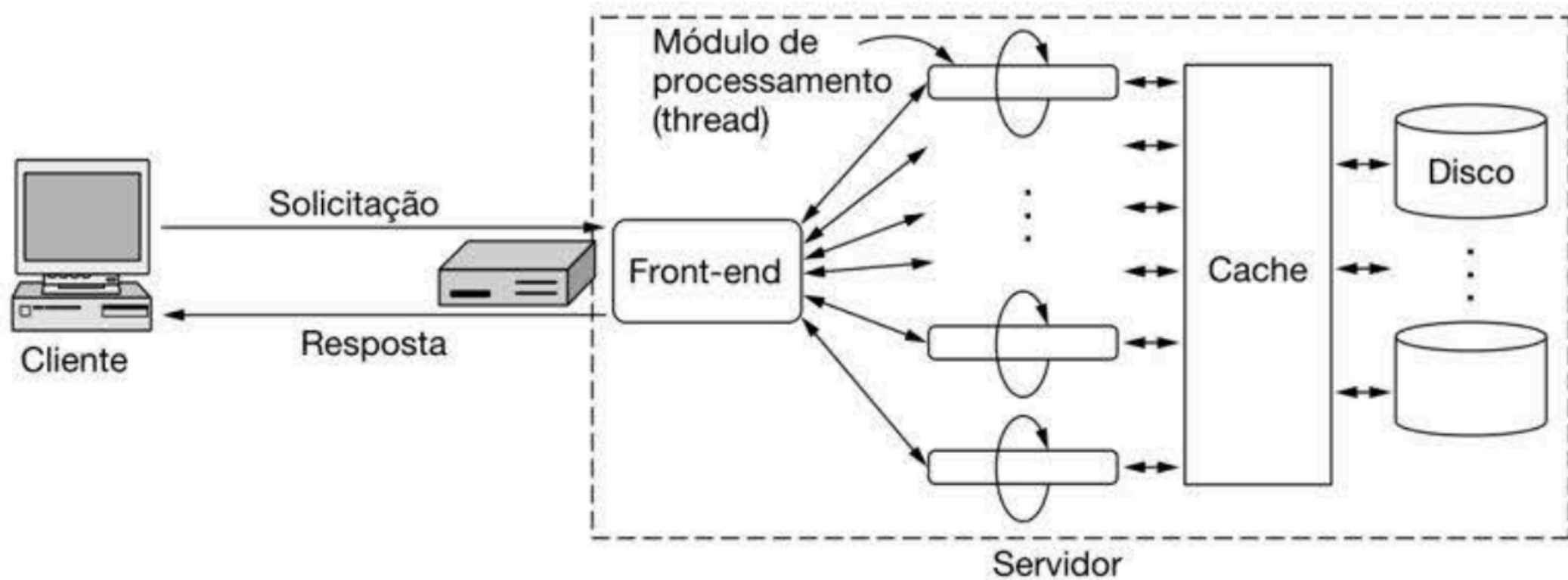
- O usuário acessa um site → Navegador (cliente) → Envia requisição HTTP → Servidor → Responde com recursos → Navegador renderiza página.
- Além de objetos estáticos como HTML, CSS e imagens, o cliente **também pode executar scripts JavaScript** para uma maior interatividade com o usuário e/ou para comunicação com o servidor.

Lado Servidor

 O servidor é responsável por processar as requisições e fornecer os recursos solicitados.

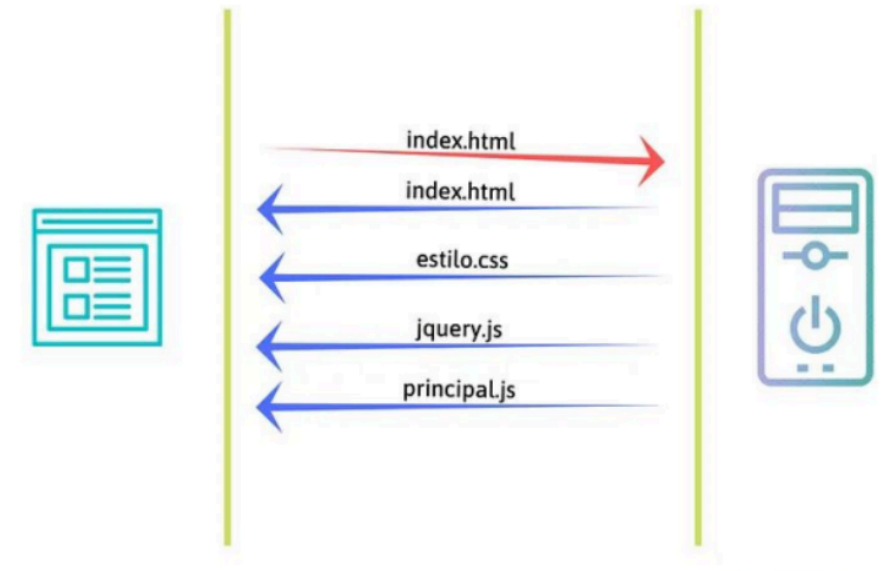
- Ele processa a requisição, acessa bancos de dados, executa lógica de negócio e retorna uma resposta adequada ao cliente.
- **Performance, segurança e escalabilidade:**
 - Utiliza cache na memória RAM para acesso instantâneo;
 - Aceita conexões e as distribui para threads de processamento;
 - Implementa mecanismos de segurança para proteger dados e recursos, que não são acessíveis no lado do cliente.

↔ Lado Cliente e Lado Servidor



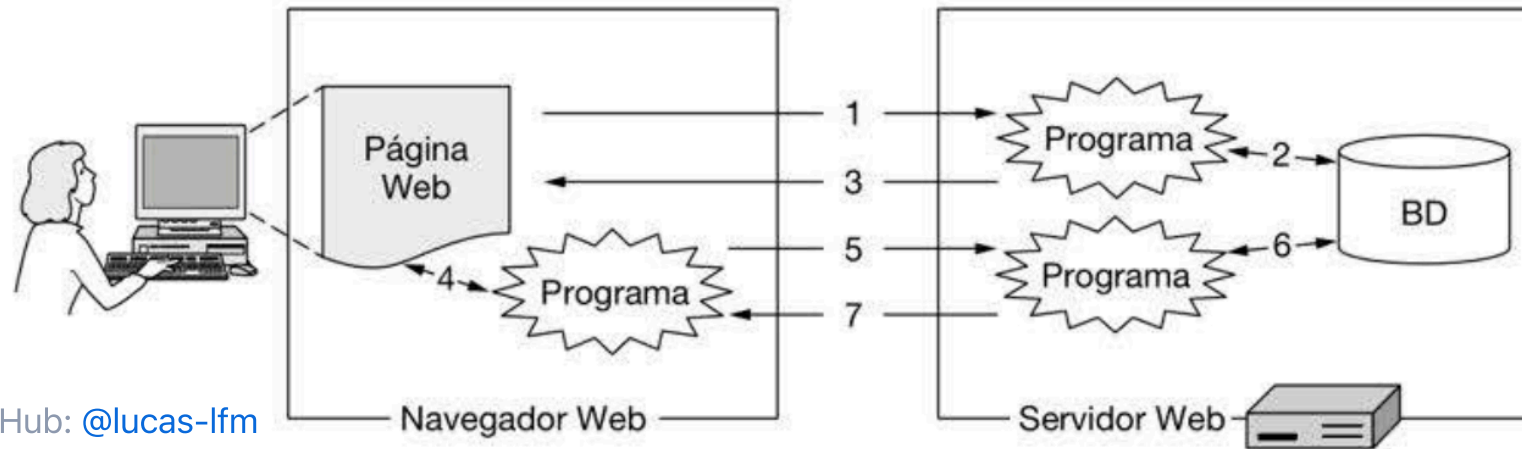
Páginas e Objetos Estáticos

- O servidor pode fornecer recursos estáticos, como: HTML, CSS, JavaScript, Imagens, Vídeos, etc.
- Recursos estáticos são recuperados do **sistema de arquivos do servidor** e entregues ao cliente, **sem necessidade de processamento adicional**.



Geração de Páginas Dinâmicas

- O servidor pode gerar páginas dinâmicas, **processando dados e lógica de negócio**.
- **Exemplo:** um site de notícias que exibe artigos com base em categorias ou preferências do usuário.
- O servidor processa a requisição, consulta um banco de dados, e gera um **HTML personalizado para o cliente**.



Frontend/Backend vs Cliente/Servidor



Frontend e Backend são termos relacionados, mas não são sinônimos de Lado Cliente e Lado Servidor.

Visão Física:

- **Lado Cliente** → Refere-se ao hardware e software na máquina do usuário final (exemplo: navegador no PC ou smartphone).
- **Lado Servidor** → Refere-se às máquinas remotas que processam e armazenam os dados e recursos da aplicação.

Frontend/Backend vs Cliente/Servidor

 **Frontend e Backend** são termos relacionados, mas não são sinônimos de **Lado Cliente** e **Lado Servidor**.

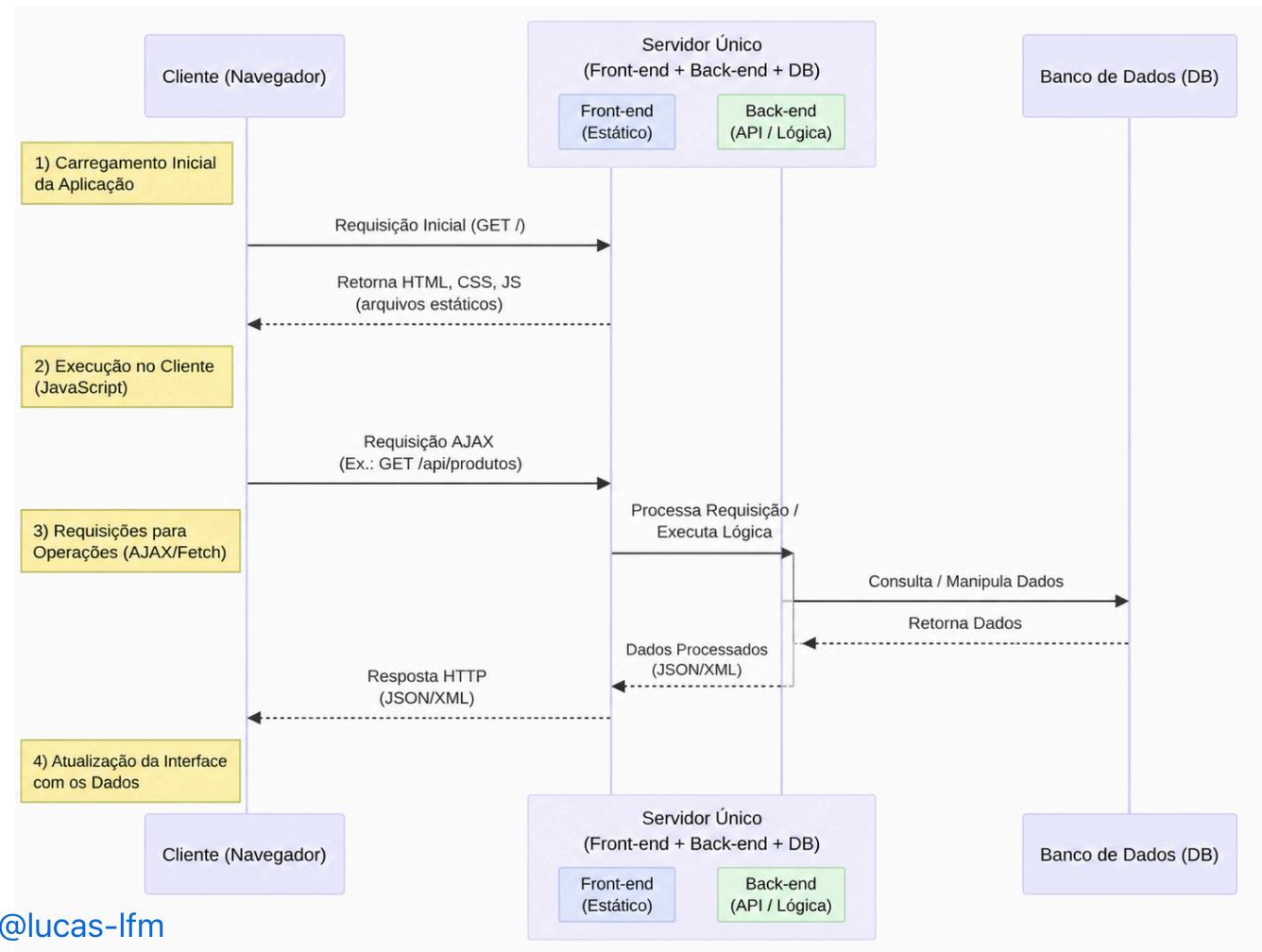
Visão Lógica:

- **Frontend** → A camada de apresentação e interface. O que o usuário vê e com o que interage diretamente, geralmente executada no lado cliente.
- **Backend** → A camada de lógica de negócio e acesso a dados. Responsável por processar as requisições e fornecer os recursos solicitados, geralmente executada no lado servidor.

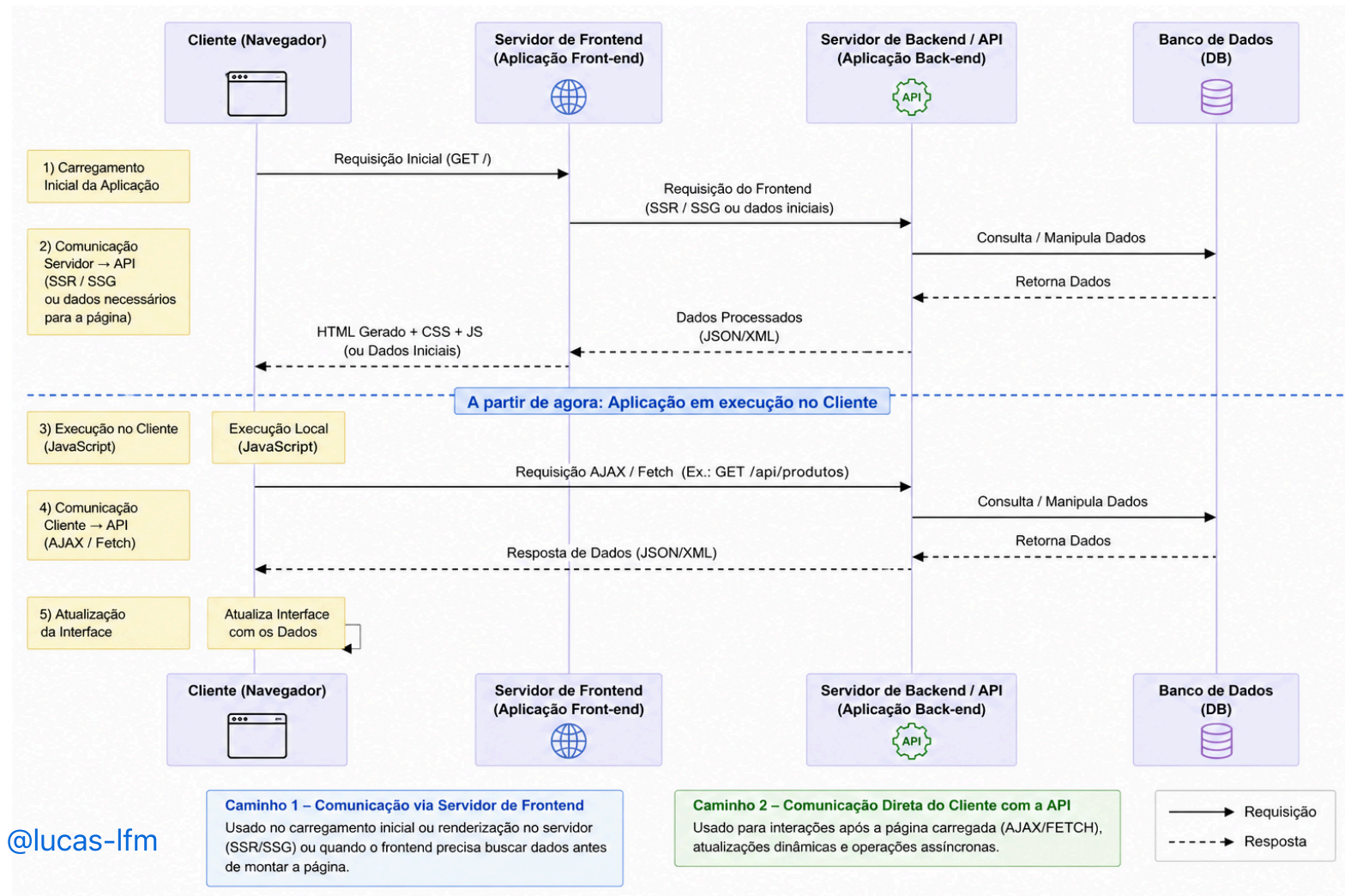
Mapeamento de Responsabilidades

Camada	Foco Principal	Local de Execução	Tecnologias (Exemplos)
Frontend	Interface e Experiência do Usuário	Lado Cliente (Navegador), Lado Servidor em alguns casos.	HTML, CSS, JavaScript
Backend	Lógica de Negócio e Dados	Lado Servidor	Java, PHP, Python, SQL, Node.js
Infraestrutura	Distribuição e Segurança	Lado Servidor (Front-end de rede)	Nginx, Proxy Reverso, Cache

Cliente → Servidor Único



Cliente → Servidor Frontend → Backend/API





Recursos Web

- Uma página web é composta por vários objetos:
 - HTML
 - CSS
 - JavaScript
 - Imagens
 - Vídeos
- Cada recurso possui um **URL único**

URL (Uniform Resource Locator)

 **Exemplo:** `http://www.exemplo.com:8080/search?q=programacao`

Estrutura:

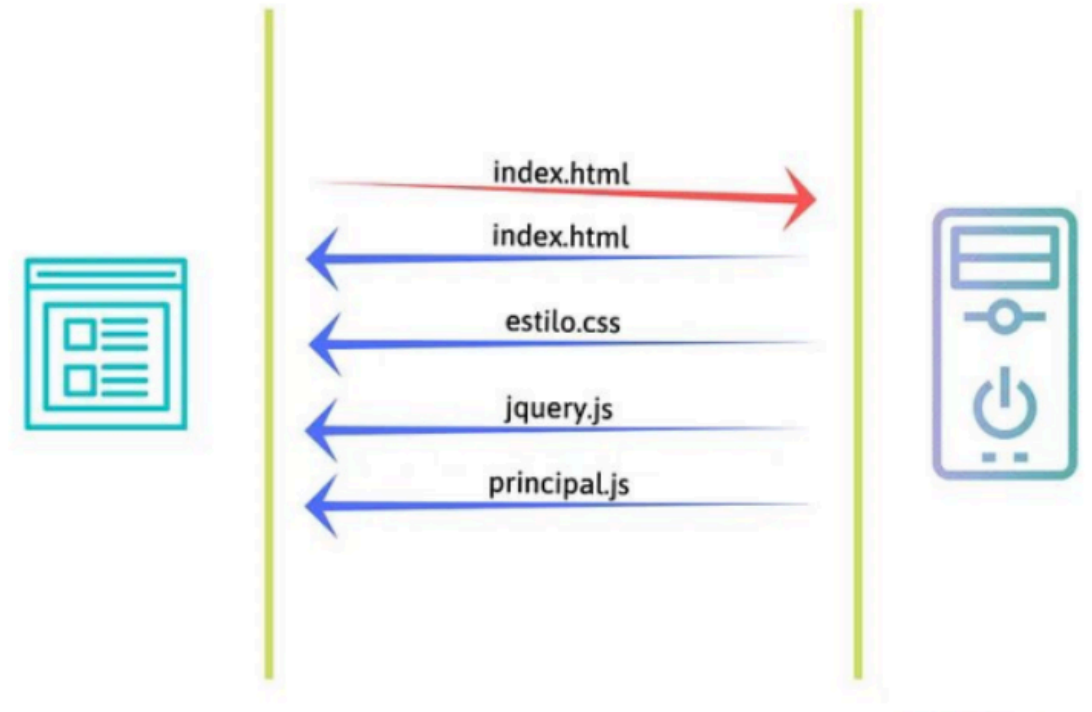
- Protocolo → `http`
- Porta → `8080`
- Domínio → `www.exemplo.com`
- Caminho → `/search`
- Query String → `?q=programacao`

Modelo Requisição-Resposta

  Comunicação básica do HTTP

Fluxo:

1. Cliente envia requisição
2. Servidor processa
3. Servidor envia resposta



Funcionamento Interno

- **HTTP utiliza TCP**
 - Porta padrão: 80 (HTTP) / 443 (HTTPS)
 - Estabelece conexão antes de enviar dados
- **Dados trafegam em texto (ASCII)**
 - Facilita depuração e leitura
 - Permite flexibilidade na estrutura das mensagens
- **HTTP é stateless**
 - Cada requisição é independente
 - Servidor não mantém estado entre requisições

Conexões HTTP

Existem dois tipos:

1. Não persistente

- Nova conexão para cada requisição

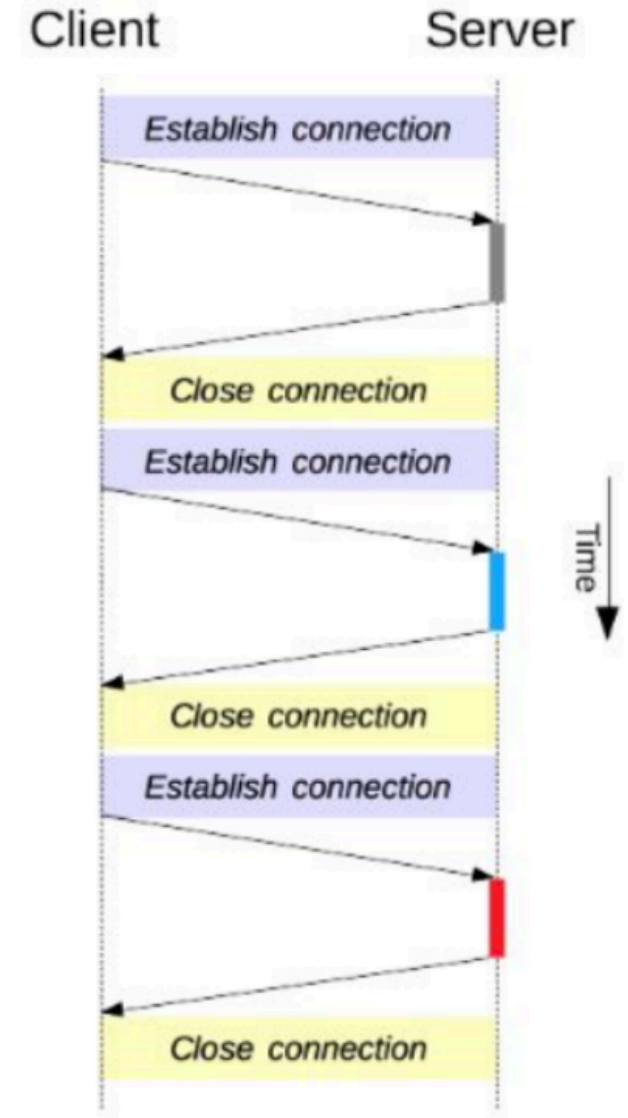
2. Persistente

- Mesma conexão para múltiplas requisições

✗ HTTP 1.0 (Não Persistente)

- Cada requisição abre uma conexão TCP
- Alto custo de comunicação
- Maior latência

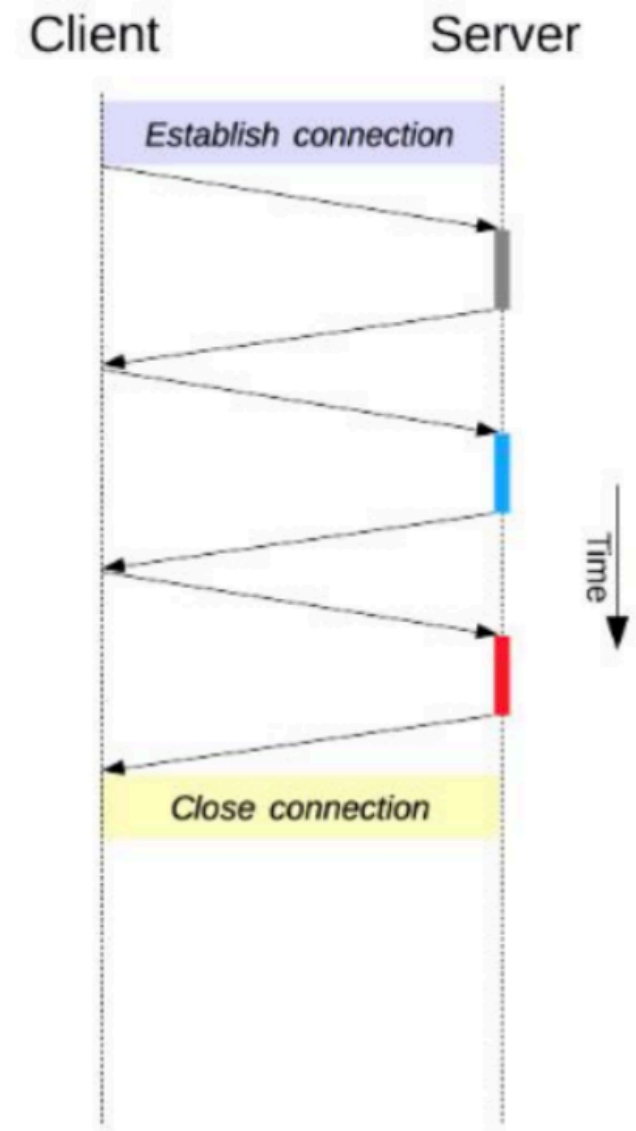
📌 Ineficiente para páginas com muitos recursos.



✓ HTTP 1.1 (Persistente)

- Mantém conexão aberta
- Permite múltiplas requisições

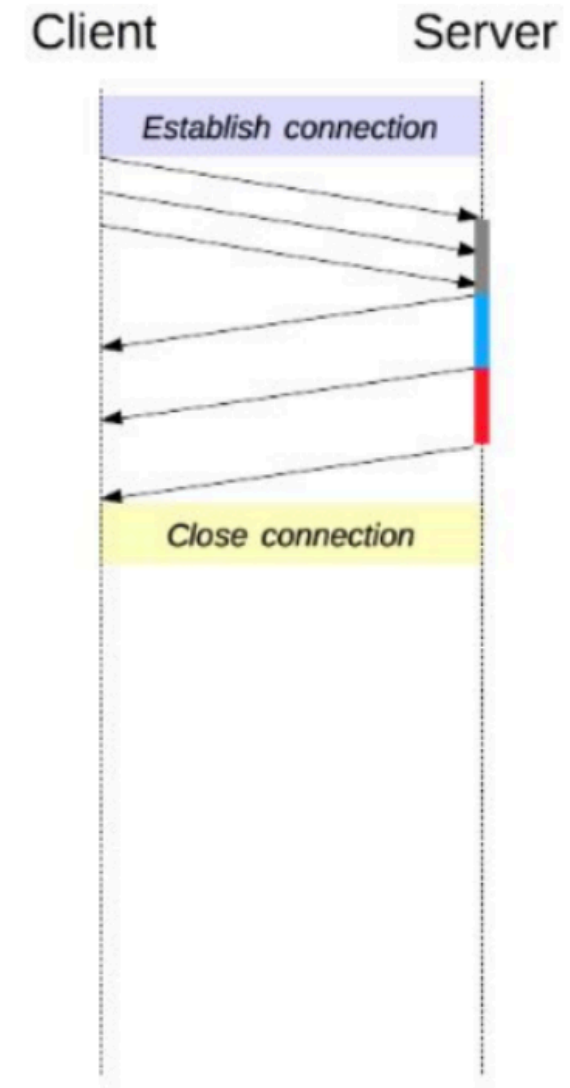
- ✓ Mais eficiente
- ✓ Menor latência



⚡ HTTP Pipelining

- Cliente envia várias requisições sem esperar respostas
- Servidor responde em sequência

Melhora desempenho da rede.



Estrutura da Requisição

Uma requisição possui:

1. Linha de requisição
2. Cabeçalhos
3. Corpo (opcional)

```
POST / HTTP/1.1
Host: localhost:8000
User-Agent: Mozilla/5.0 (Macintosh;... )... Firefox/51.0
Accept: text/html,application/xhtml+xml,..., */*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Content-Type: multipart/form-data; boundary=-12656974
Content-Length: 345

-12656974
(more data)
```

Request headers

General headers

Entity headers

Linha de Requisição

Contém:

- Método HTTP
- URL
- Versão do protocolo

Exemplo: `GET /index.html HTTP/1.1`



Métodos HTTP

i Os métodos indicam a **ação desejada** sobre um recurso.

Principais:

- **GET** → obter recurso
- **POST** → enviar dados (criar recurso)
- **PUT** → atualizar recurso
- **DELETE** → remover recurso

Método	Descrição
GET	Lê uma página Web
HEAD	Lê um cabeçalho de página Web
POST	Acrescenta algo a uma página Web
PUT	Armazena uma página Web
DELETE	Remove a página Web
TRACE	Ecoa a solicitação recebida
CONNECT	Conecta através de um proxy
OPTIONS	Consulta opções para uma página



Cabeçalhos HTTP

Formato: Nome: Valor

Exemplos:

- Host
- User-Agent
- Content-Type



Fornecem metadados da requisição.

Corpo da Requisição

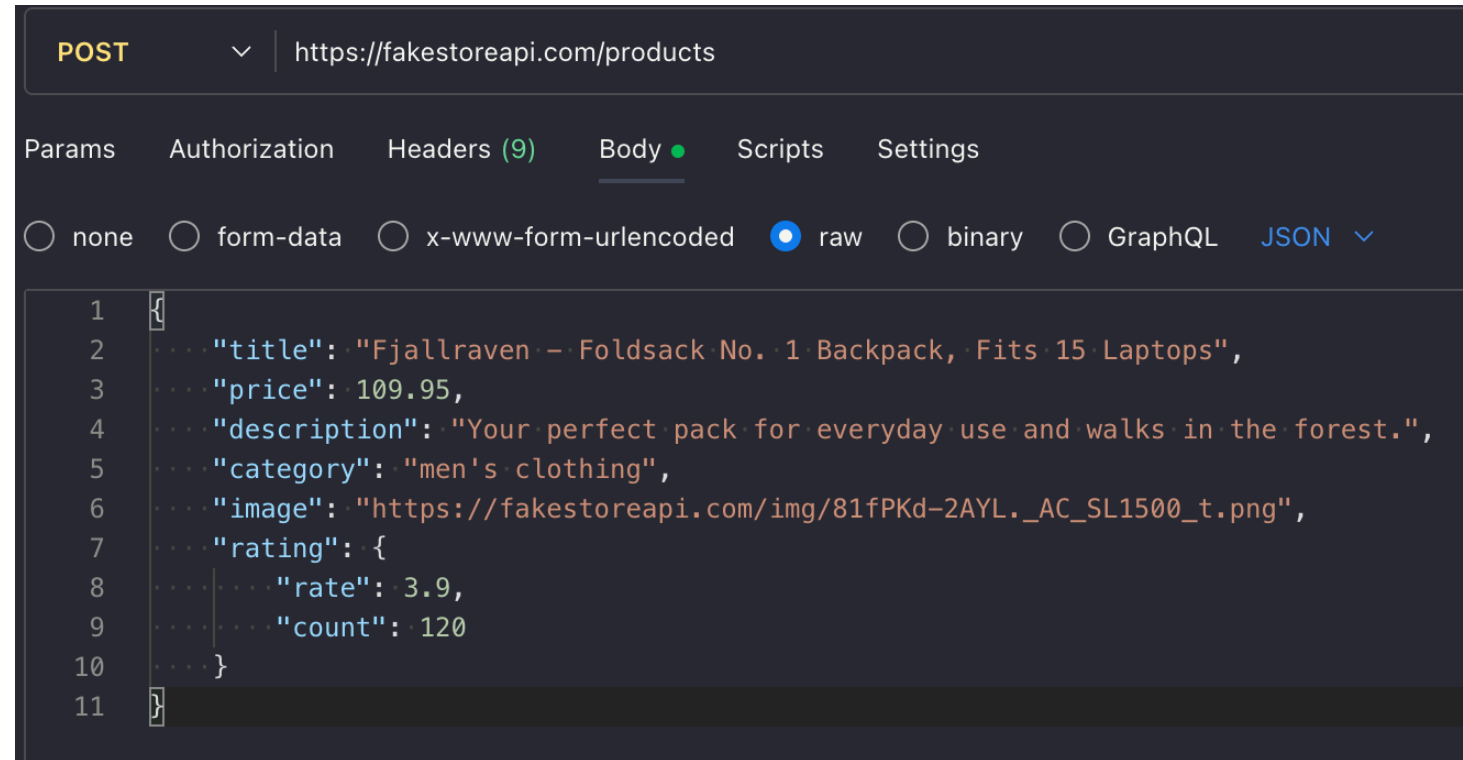
Presente em:

- **POST**
- **PUT**

Contém dados enviados ao servidor.

Exemplo:

- JSON
- Formulários



```
POST https://fakestoreapi.com/products

Params  Authorization  Headers (9)  Body  Scripts  Settings

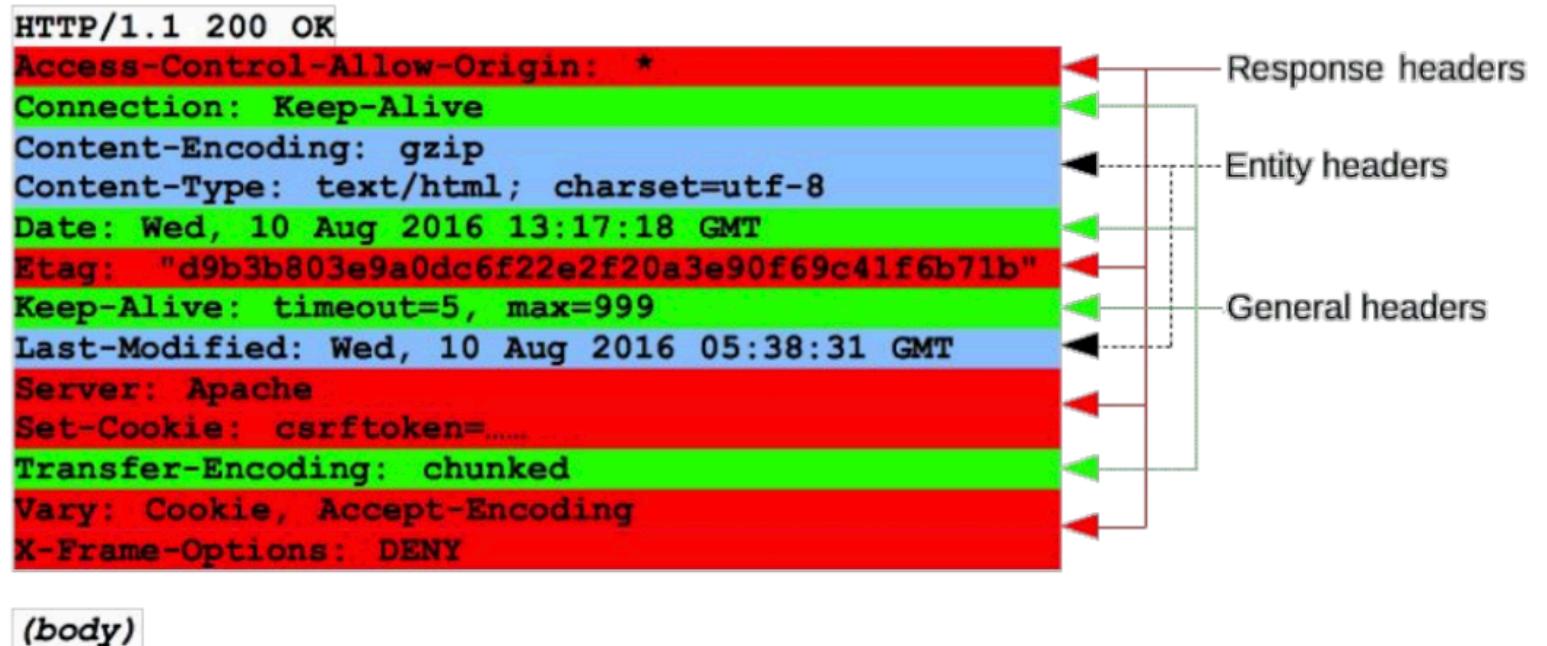
none  form-data  x-www-form-urlencoded  raw  binary  GraphQL  JSON

1 {
2   "title": "Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops",
3   "price": 109.95,
4   "description": "Your perfect pack for everyday use and walks in the forest.",
5   "category": "men's clothing",
6   "image": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_t.png",
7   "rating": {
8     "rate": 3.9,
9     "count": 120
10  }
11 }
```

Estrutura da Resposta

Uma resposta possui:

1. Linha de status
2. Cabeçalhos
3. Corpo





Linha de Status

Contém:

- Versão HTTP
- Código de status
- Mensagem descritiva

Exemplo: `HTTP/1.1 200 OK`

Códigos de Status

Indicam resultado da requisição e se dividem em:

- **2xx** → Sucesso
- **3xx** → Redirecionamento
- **4xx** → Erro do cliente
- **5xx** → Erro do servidor

Definidos na RFC 9110:

<https://datatracker.ietf.org/doc/html/rfc9110>.

Código	Significado	Exemplos
1xx	Informação	100 = servidor concorda em tratar da solicitação do cliente
2xx	Sucesso	200 = solicitação com sucesso; 204 = nenhum conteúdo presente
3xx	Redirecionamento	301 = página movida; 304 = página em cache ainda válida
4xx	Erro do cliente	403 = página proibida; 404 = página não localizada
5xx	Erro do servidor	500 = erro interno do servidor; 503 = tente novamente mais tarde



Corpo da Resposta

Conteúdo retornado pelo servidor:

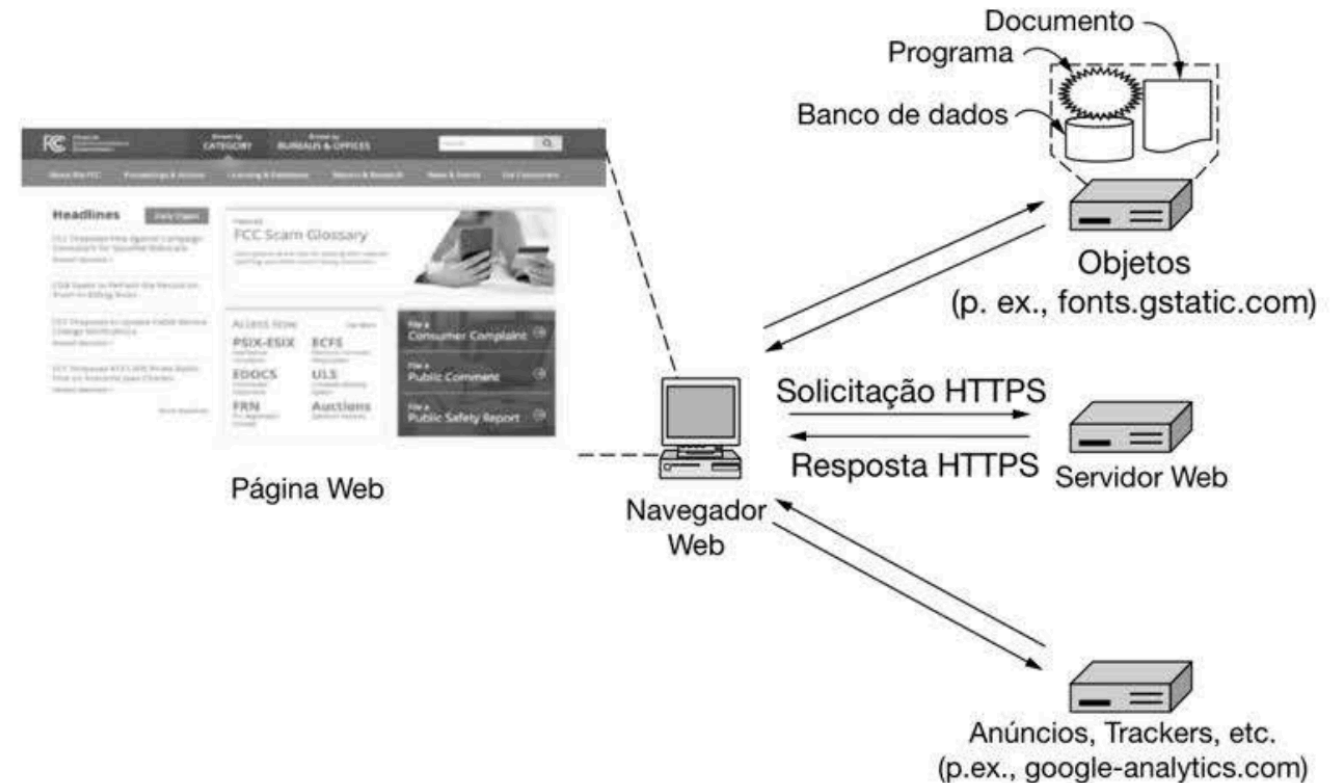
- HTML
- JSON
- Imagens

✓ Base da renderização no navegador

HTTP na Prática

Exemplo real:

1. Usuário acessa site
2. Browser envia GET
3. Servidor responde HTML
4. Browser solicita recursos adicionais





Atividade em Sala (30 min)



Objetivo

Analisar requisições HTTP reais



Atividade

1. Acesse um site qualquer
2. Abra o DevTools (F12)
3. Vá na aba **Network**

Tarefa

Para uma requisição:

- Identifique:
 - Método HTTP
 - URL
 - Status code
 - Headers
 - Response

Desafio

- Compare duas requisições:
 - HTML vs API (JSON)
- Identifique diferenças

Reflexão

- Como o HTTP impacta:
 - Performance?
 - Segurança?
 - Escalabilidade?

? Dúvidas

- Perguntas?
- Discussões?

Referências



Próximas Aulas

- Introdução à Linguagem Java
- Java para Web: Servlets e JSP